Fundamentals of Data Science

Lecture 7 - Statistical Modelling Part II

Adrian Ochs [1] and Christian Roerig [2]

Michaelmas 2025

[1] Co-founder Quantum Leap, ex-QuantCo

[2] Data Scientist/ML Engineer @ QuantCo

Table of contents

Model accuracy

Model accuracy

There are multiple metrics to consider, when evaluating the accuracy of a model:

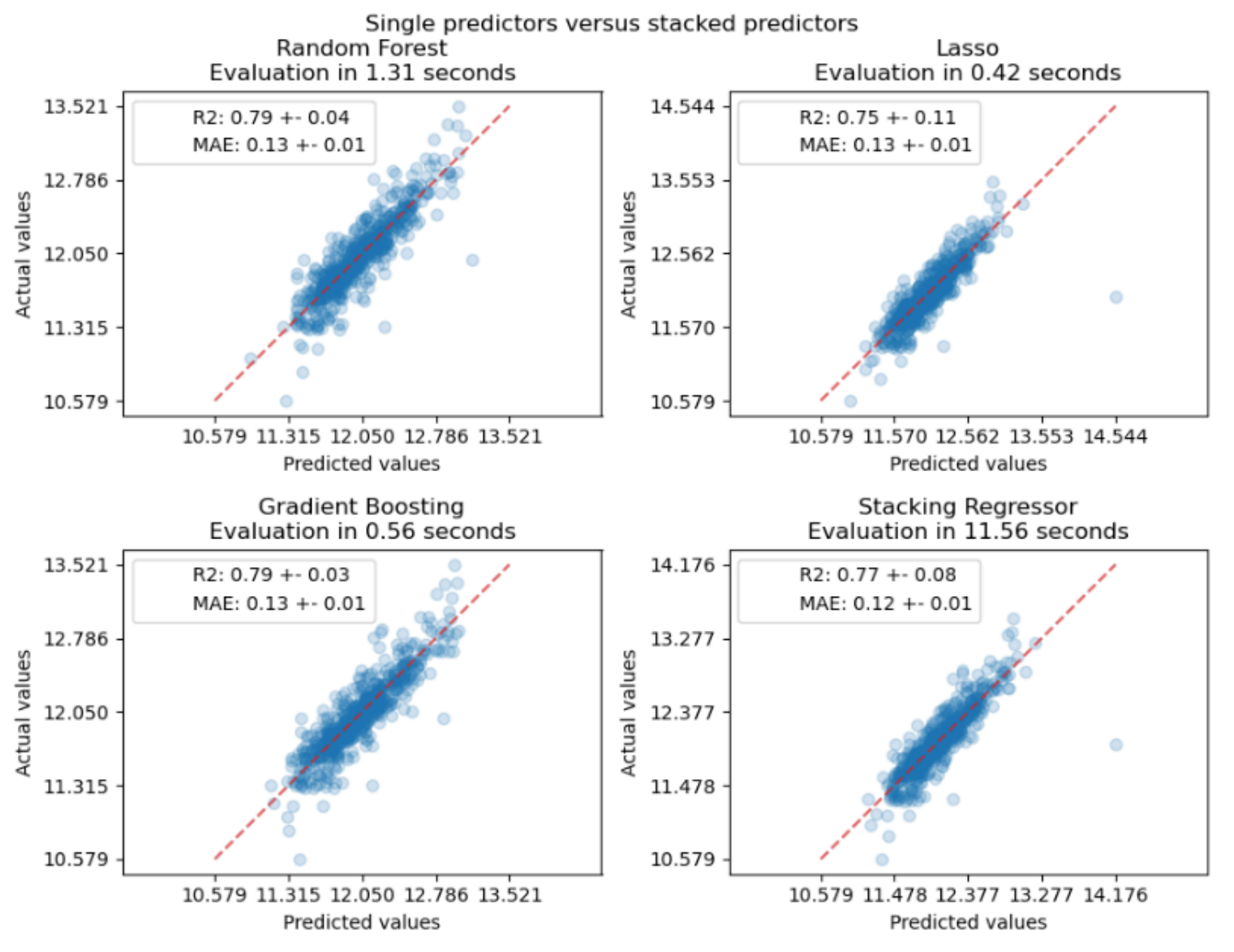
- **Bias:** We want as little bias as possible in our estimates (bias-variance-tradeoff).
- **Validation error:** E.g. Deviance, MSE, MAE, MSPE.
- **Ranking of predictions:** The ranking of predictions should be as close as possible with the true values.

Bias

One of the first checks usually is, whether the predicted means on the training set are correct. The simplest way is to compare the (weighted) mean predictions with the (weighted) mean outcomes.

While we might have no substantial bias on average, we might also want to check for biases along the distribution of our predictions, as we might over-/under-estimate low outcomes and under-/over-estimate high outcomes.

Predicted vs. Actual values



Source: Scikit-Learn example

Deviance

It's essentially a generalization of the sum of squared residuals for maximum likelihood estimation (e.g. GLM):

$$D(y, \hat{\mu}) = 2(log(p(y|\hat{\theta}_s)) - log(p(y|\hat{\theta}_0))),$$

where $\hat{\theta}_0$ are the fitted parameters and $\hat{\theta}_s$ are the parameters of a **saturated** model, i.e. a model with a parameter for each observation, with a perfect fit. Consequently, the closer we are to the perfect fit, the lower the deviance, the better.

We usually report the mean deviance over all observations in the sample.

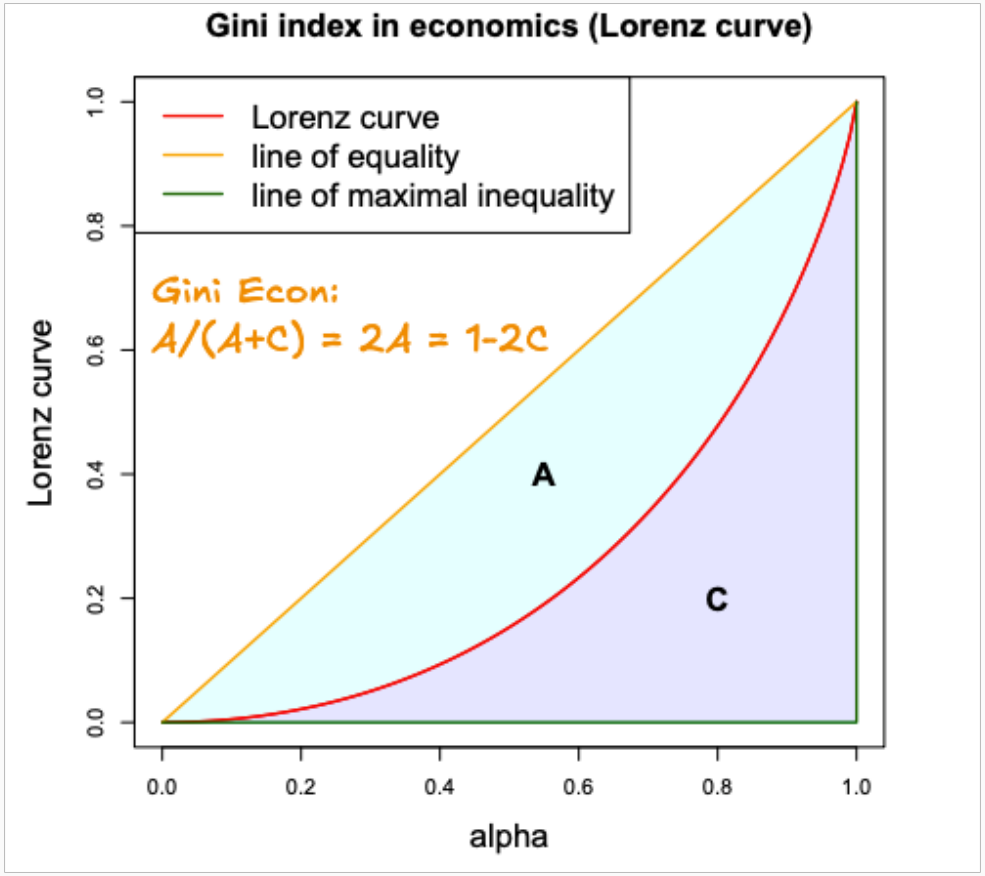
Note

The **log-likelihood of the saturated** (perfect) model does not have to be 0. It depends on the distribution of the data and the underlying distribution assumption. There are exceptions for which it is the case, e.g. binary outcomes and binomial distribution. The **deviance of the saturated model** is of course 0.

Ranking of predictions - Normalized Gini

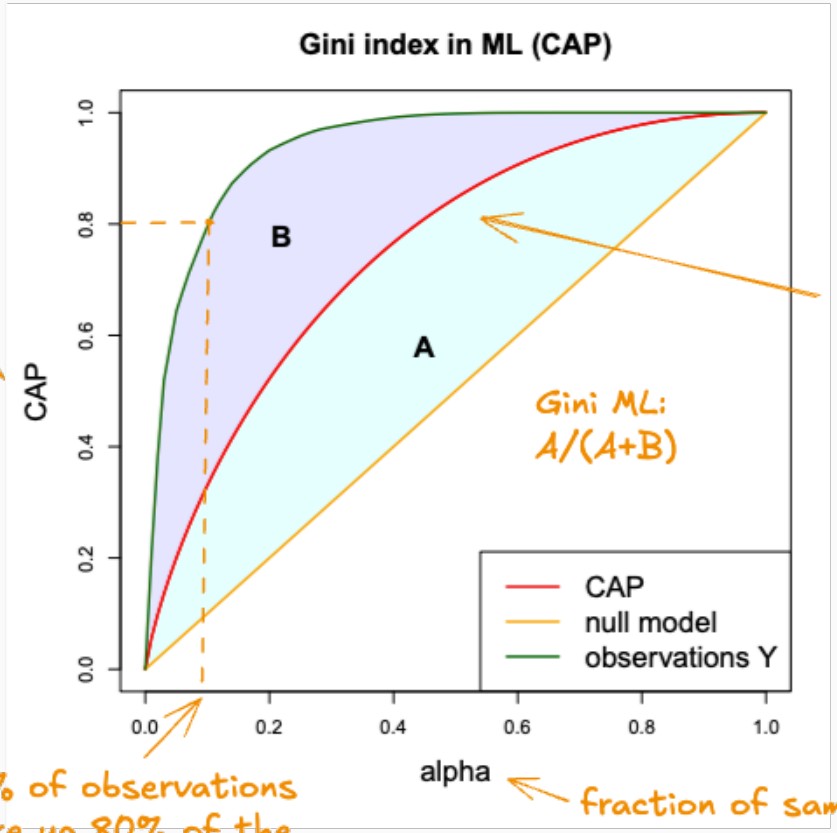
The Normalized Gini tells us how well the model sorts the observations.

We know the Gini Index (or Lorenz curve) from Economics when talking about inequality:



Ranking of predictions - Normalized Gini

In Machine Learning, we consider a slightly different version, as we are not interested in the inequality per se, but how well, the predictions match the actual ranking of observations:



Cumulative Accuracy Profile

the model fits the dispersion in the data to some degree but is far of from the actual dispersion/inequality in the data

10% of observations make up 80% of the sum of the target variable

fraction of sample

Regularization

Regularization

- **Explicit regularization:** Is when we explicitly add a term to the optimization problem, via a constraint or penalty (e.g. L1/L2-penalty).
- **Implicit regularization:** Encompasses all other forms of regularization via learning parameters (e.g. tree hyperparameters), early stopping. This is happening whenever we tune ML approaches.

Shrinkage methods

Shrinking coefficient estimates of parametric models, can significantly reduce their variance and improve out-of-sample performance usually at a relatively small cost of bias (bias-variance-tradeoff).

- Ridge (or Tikhonov) regularization:

$$\sum_{i}^{n}(y_i - \beta_0 - \sum_{j=1}^{p}\beta_j x_{ij})^2 + \lambda \sum_{j=1}^{p}\beta_j^2$$

- LASSO (Least Absolute Shrinkage and Selection Operator):

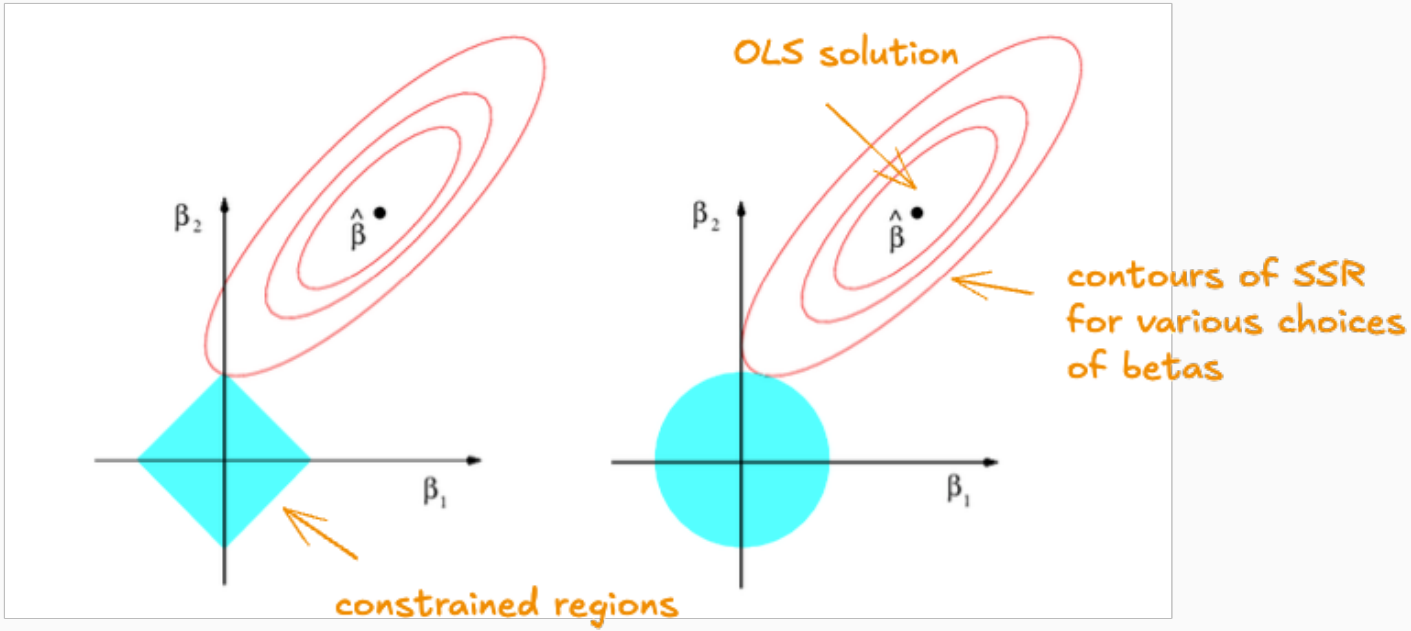
$$\sum_{i}^{n}(y_i - \beta_0 - \sum_{j=1}^{p}\beta_j x_{ij})^2 + \lambda \sum_{j=1}^{p}|\beta_j|$$

- Elastic net: Linear combination of L1 and L2-penalty terms.

Shrinkage methods

The LASSO and the Ridge regression can also be expressed as constrained optimization problems with some bound for penalty term s. For every λ, we can find a corresponding s, which gives the same coefficients:

$$min_\beta \sum_i^n (y_i - \beta_0 - \sum_{j=1}^p \beta_j x_{ij})^2 \quad s.t. \quad \sum_{j=1}^p ||\beta_j|| \leq s$$



Source: ISL, chapter 6

Feature Selection

Feature Selection

Most of the time it doesn't make sense to simply throwing all available features in the model (multicollinearity, irrelevance, compute resources), so we have to decide which features to use.

We will consider the following approaches:

- Forward stepwise selection
- Feature selection via LASSO
- Maximum relevance minimum redundancy (MRMR)
- Tree-based feature selection
- Permutation feature importance

Forward stepwise selection

In principle, we could compute models for all feature combinations and then select the best model (*best subset selection*). But this would mean 2^p models, for p predictors, which is typically infeasible.

A computationally more efficient option is to forward select features.

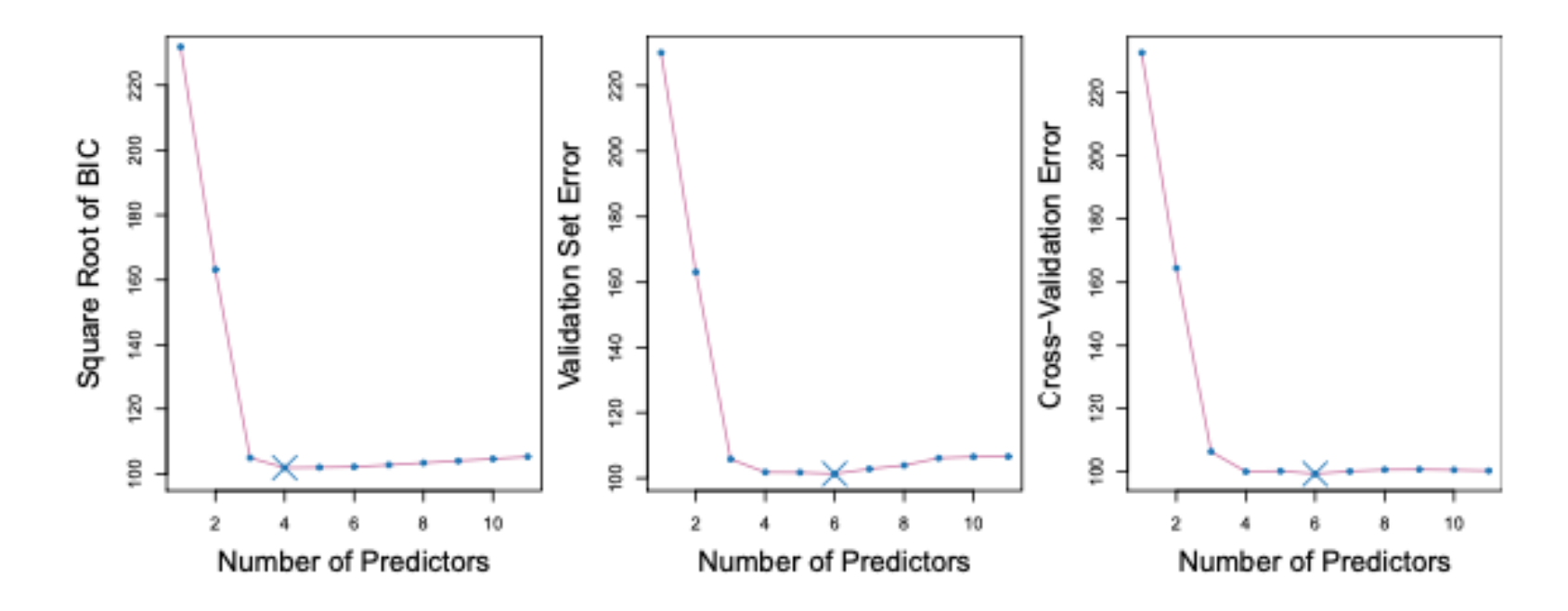
Algorithm 6.2 *Forward stepwise selection*

1. Let $\mathcal{M}_0$ denote the *null* model, which contains no predictors.
2. For $k = 0, \ldots, p-1$:
   (a) Consider all $p - k$ models that augment the predictors in $\mathcal{M}_k$ with one additional predictor.
   (b) Choose the *best* among these $p - k$ models, and call it $\mathcal{M}_{k+1}$. Here *best* is defined as having smallest RSS or highest R^2.
3. Select a single best model from among $\mathcal{M}_0, \ldots, \mathcal{M}_p$ using the prediction error on a validation set, C_p (AIC), BIC, or adjusted R^2. Or use the cross-validation method.

Source: ISL, chapter 6

Forward stepwise selection

The different criteria (R2, BIC, AIC) could suggest different numbers of predictors, yet it is recommended to use the cross-validation error as it makes fewer assumptions about the underlying model.



Source: ISL, chapter 6

LASSO feature selection

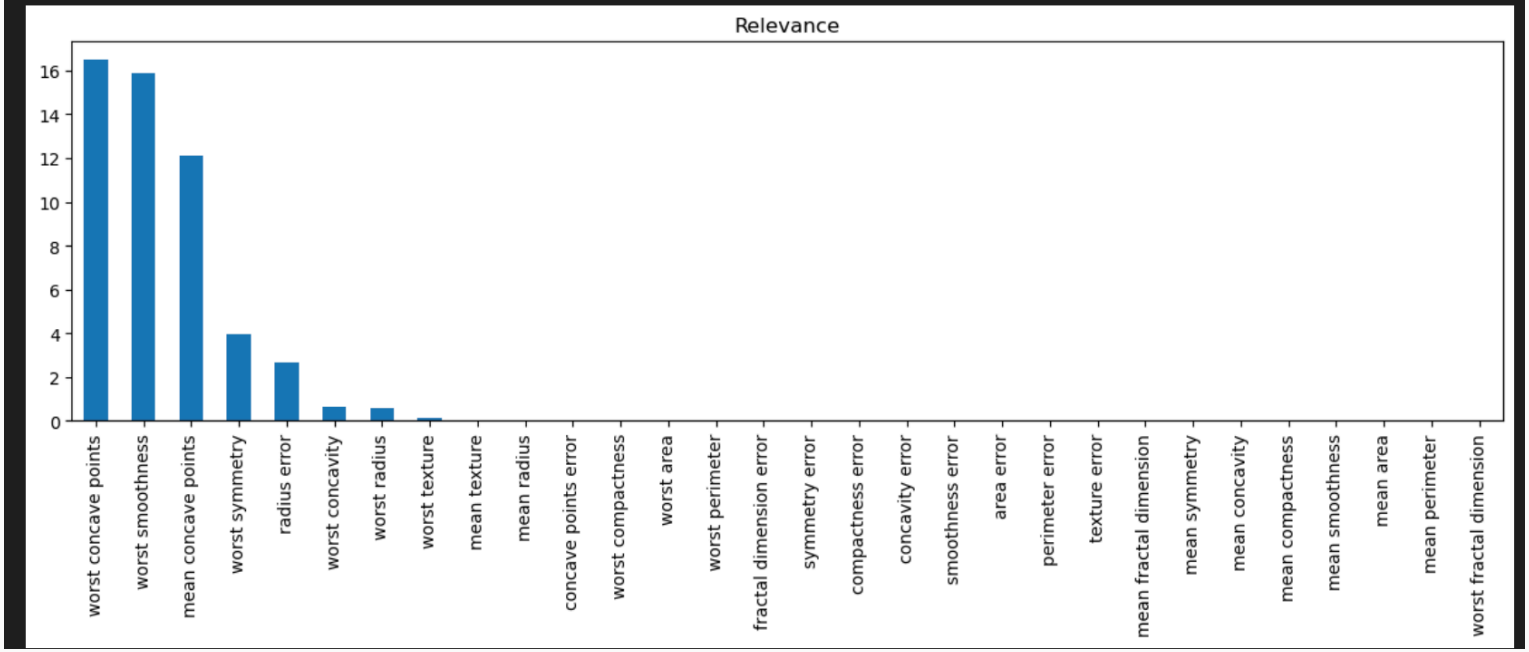
We can also exploit the sparse solutions of L1-regularization for feature selection and simply drop the features which were shrunk to zero by the penalty term.

The higher the weight on the penalty term (often referred to as α or λ), the less features are selected.

LASSO feature selection

```python
import pandas as pd
import matplotlib.pyplot as plt
from sklearn.datasets import load_breast_cancer
from sklearn.model_selection import train_test_split
from glum import GeneralizedLinearRegressor

X, y = load_breast_cancer(return_X_y=True, as_frame=True)

sel = GeneralizedLinearRegressor(family="binomial", alpha=0.01, l1_ratio=1, scale_predictors=True)
sel.fit(X, y)

pd.Series(abs(sel.coef_), index=sel.term_names_).sort_values(
    ascending=False).plot.bar(figsize=(15, 4))
plt.title("Relevance")
plt.show()
```

✓ 0.2s



Maximum Relevance Minimum Redundancy (MRMR)

The MRMR algorithm stems from bioinformatics as a method to select features for microarray gene expression data, but was popularized by researchers at UBER: Uber MRMR paper.

The idea is relatively simple: We want the most relevant features with the lowest redundancy to other features in the model.

Both, for relevance and redundancy, there are several way to measure them and relate them. More details: MRMR implementation in Feature-engine.

MRMR

The MRMR comes in different flavours for the relevance and redundancy measures and can be expressed in differences or ratios:

The FCD (F-test correlation difference) uses the F-statistic to score the relevance, and correlation to score the redundancy:

$$f^{FCD}(X_i) = F(Y, X_i) - \frac{1}{|S|} \sum_{X_s \in S} \rho(X_s, X_i), \quad (6)$$

where $\rho(X_s, X_i)$ is the Pearson correlation, and $F(Y, X_i)$ is the F-statistic.

Similarly, the FCQ (F-test correlation quotient) uses the quotient scheme:

$$f^{FCQ}(X_i) = F(Y, X_i) / [\frac{1}{|S|} \sum_{X_s \in S} \rho(X_s, X_i)]. \quad (7)$$

feature importance score for candidate X_i

S is set of features already selected into the model

here, redundancy is measured as the Pearson correlation with all S features already in the model. We use correlation rather than e.g. covariance to have a comparable scale.

for the relevance we use the F-test statistic here, between the target variable and the candidate

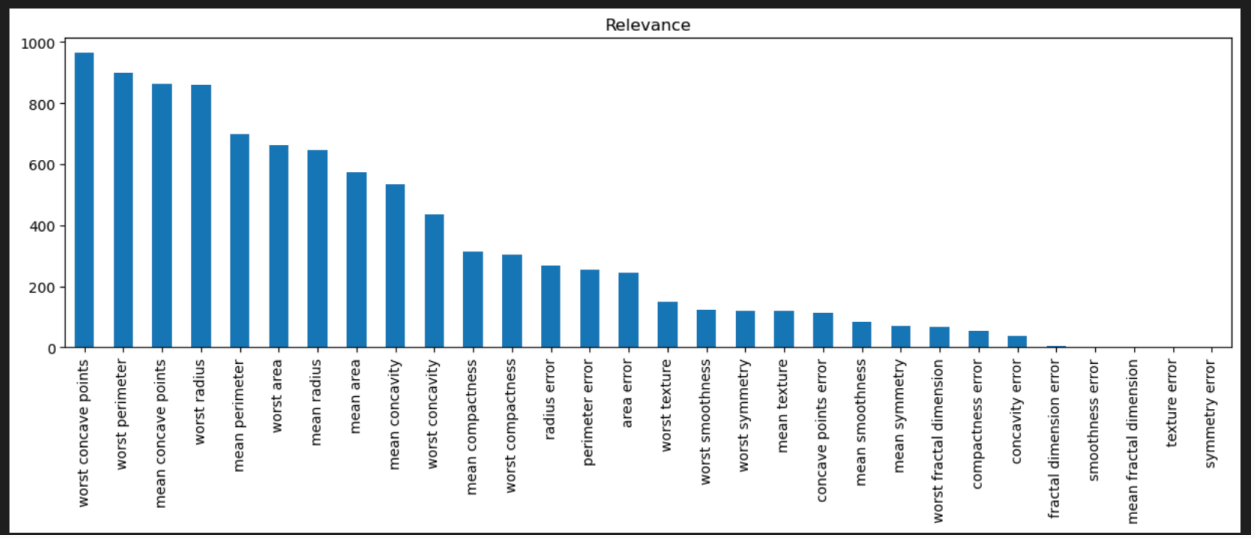
Excerpt from Uber MRMR paper

MRMR

While, the MRMR algorithm sorts the features for us, given their MRMR score, we have to decide on a number of features to include. Typically, when plotting the MRMR score we see a kink somewhere, after which the features don't show any meaningful score.

```python
import pandas as pd
import matplotlib.pyplot as plt
from sklearn.datasets import load_breast_cancer
from sklearn.model_selection import train_test_split
from feature_engine.selection import MRMR

X, y = load_breast_cancer(return_X_y=True, as_frame=True)

sel = MRMR(method="FCQ")
sel.fit(X, y)

pd.Series(sel.relevance_, index=sel.variables_).sort_values(
    ascending=False).plot.bar(figsize=(15, 4))
plt.title("Relevance")
plt.show()
```

✓ 0.2s



Tree-based feature selection

Tree based methods provide valuable feature importance information based on the splits they computed. Based on the feature importance, we can discard irrelevant features.

Note

The number of splits as a measure of feature importance is biased towards high-cardinality features as we have to split more often than for low-dimensional features. The split gain, i.e. the improvement in the loss function after each split, is more informative about the actual feature importance.

Permutation feature importance

One model agnostic way of computing the feature importance, is by randomly shuffling the feature and observe the degradation of the model's score. Those features which lead to a higher loss in predictive accuracy, when shuffled, are more important.

- Inputs: fitted predictive model m, tabular dataset (training or validation) D.
- Compute the reference score s of the model m on data D (for instance the accuracy for a classifier or the R^2 for a regressor).
- For each feature j (column of D):
  - For each repetition k in $1, \ldots, K$:
    - Randomly shuffle column j of dataset D to generate a corrupted version of the data named $\tilde{D}_{k,j}$.
    - Compute the score $s_{k,j}$ of model m on corrupted data $\tilde{D}_{k,j}$.
  - Compute importance i_j for feature f_j defined as:

$$i_j = s - \frac{1}{K} \sum_{k=1}^{K} s_{k,j}$$

Source: Scikit-learn docs

Hyperparameter Tuning

Important hyperparameters for GBMs

Besides the `learning_rate` and the number of estimators `n_estimators`, we also have to tune the complexity of the individual trees. The most important learning parameters for leaf-wise growth are:

- `num_leaves`: This is the main parameter to control the complexity of the tree model. The fewer leaves we allow, the less deep will the tree be. For depth-wise growth we would use `max_depth`.
- `min_data_in_leaf`: The minimum amount of observations we want in each leaf. This is a very important parameter to prevent over-fitting in a leaf-wise tree.

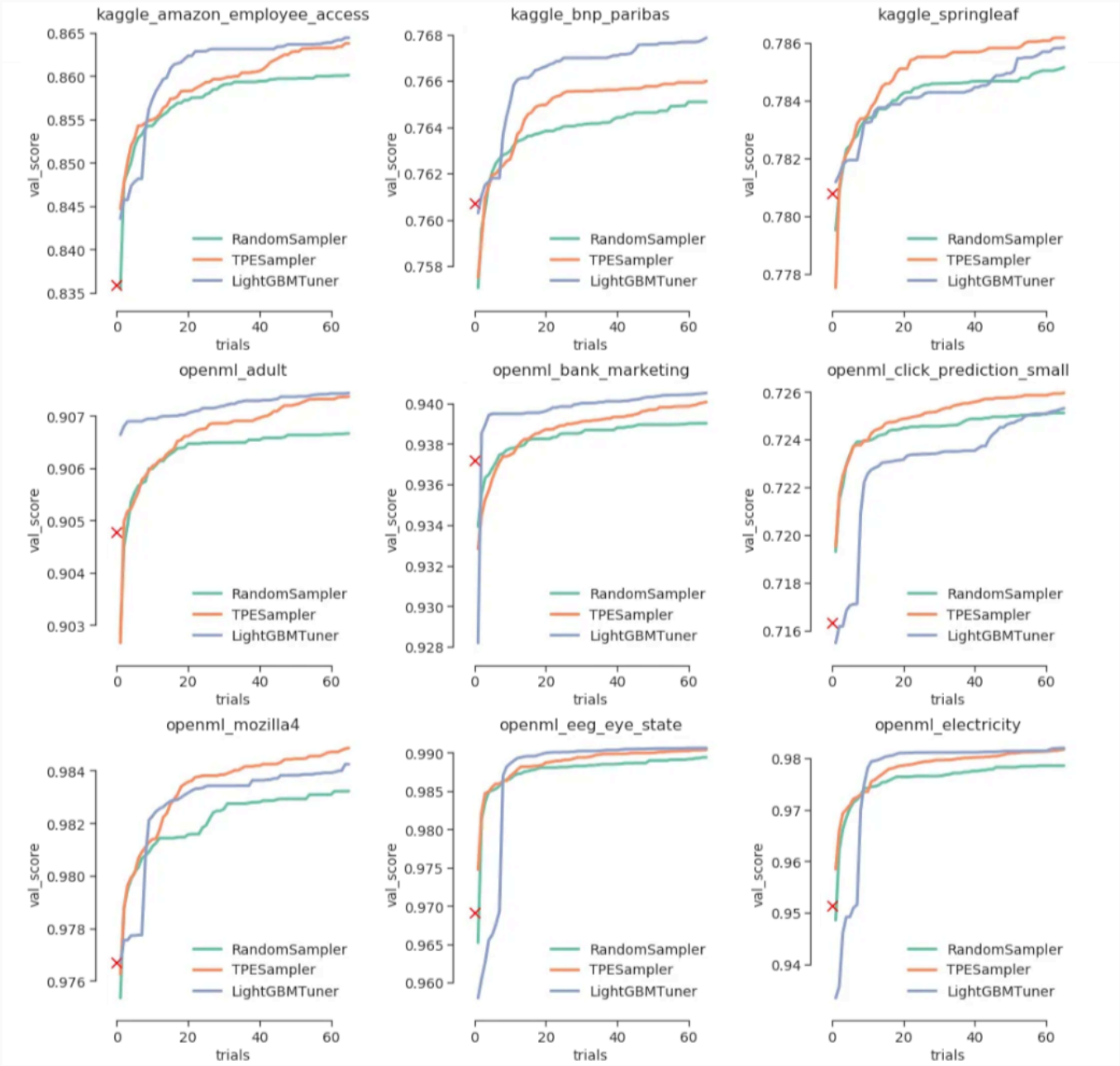
For more insights on specific parameters, see: LGBM docs on parameter tuning.

Optuna package

The main challenge with hyperparameter tuning is to get better evaluations with as little tuning runs as possible. We have seen the `GridSearchCV` approach in the last lecture, which would try all combinations of grid points which quickly evaluates to many trials. Instead, we can use step-wise tuning, based on the learning behaviour of specific algorithms. This is what the LGBM Tuning implementation in optuna does.

For details, see: LGBM Tuner in optuna.

LGBM Tuner


kaggle_amazon_employee_access
kaggle_bnp_paribas
kaggle_springleaf
openml_adult
openml_bank_marketing
openml_click_prediction_small
openml_mozilla4
openml_eeg_eye_state
openml_electricity
val_score
trials
RandomSampler
TPESampler
LightGBMTuner

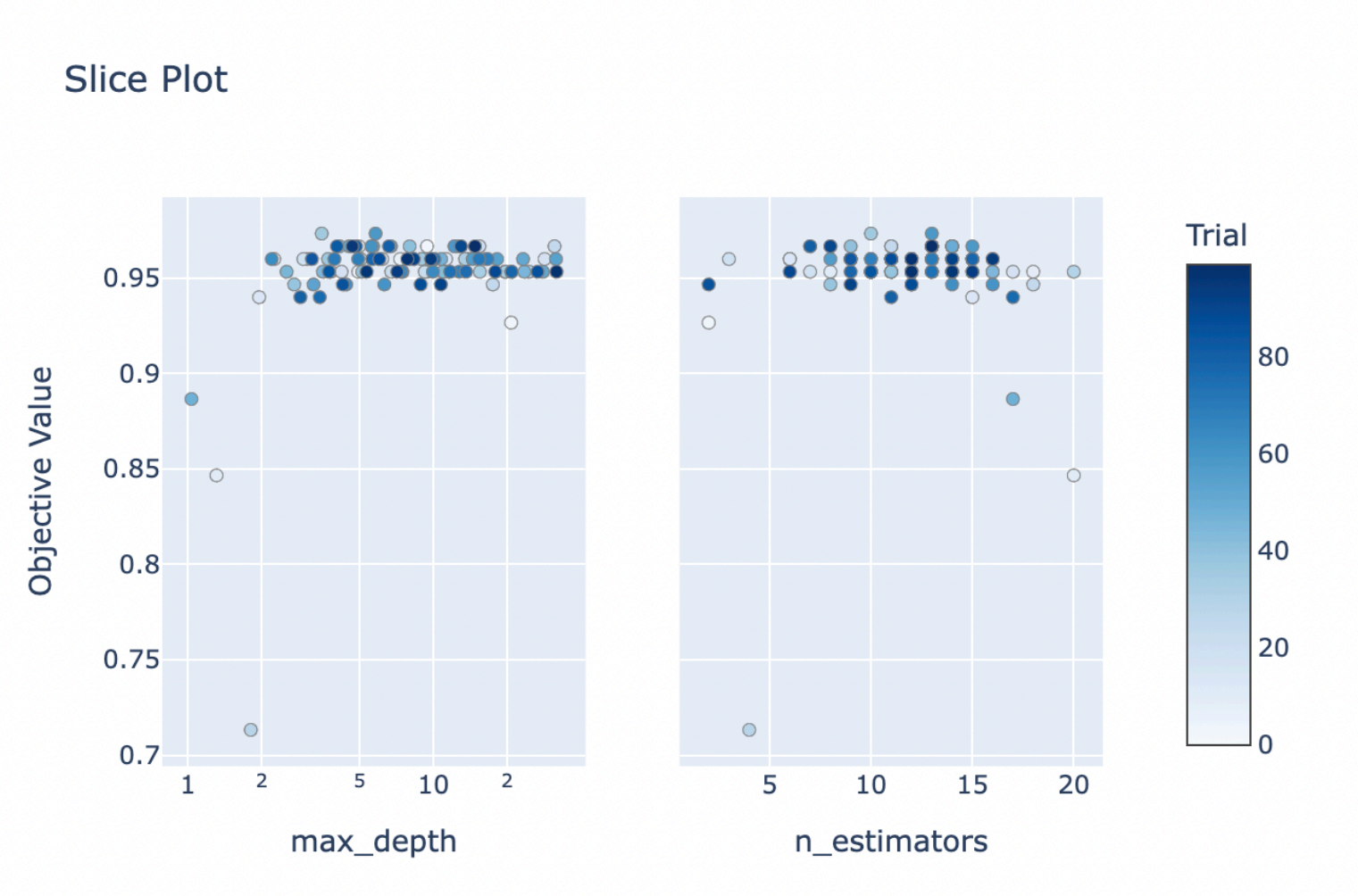
Optuna package - example

```python
import optuna
import sklearn.datasets
import sklearn.ensemble
import sklearn.model_selection

def objective(trial):
    iris = sklearn.datasets.load_iris()

    n_estimators = trial.suggest_int("n_estimators", 2, 20)
    max_depth = int(trial.suggest_float("max_depth", 1, 32, log=True))

    clf = sklearn.ensemble.RandomForestClassifier(n_estimators=n_estimators, max_depth=max_depth)

    return sklearn.model_selection.cross_val_score(
        clf, iris.data, iris.target, n_jobs=-1, cv=3
    ).mean()


study = optuna.create_study(direction="maximize")
study.optimize(objective, n_trials=100)

trial = study.best_trial

print("Accuracy: {}".format(trial.value))
print("Best hyperparameters: {}".format(trial.params))
```

✓ 8.4s

```
[I 2024-10-21 23:01:32,529] A new study created in memory with name: no-name-6147928a-88da-4f11-b518-d1619fcbbc3c
[I 2024-10-21 23:01:33,678] Trial 0 finished with value: 0.9733333333333333 and parameters: {'n_estimators': 12, 'max_depth': 4.3556
[I 2024-10-21 23:01:34,374] Trial 1 finished with value: 0.9533333333333333 and parameters: {'n_estimators': 7, 'max_depth': 2.04851
[I 2024-10-21 23:01:35,034] Trial 2 finished with value: 0.9466666666666667 and parameters: {'n_estimators': 14, 'max_depth': 1.3369
[I 2024-10-21 23:01:35,776] Trial 3 finished with value: 0.9533333333333333 and parameters: {'n_estimators': 18, 'max_depth': 4.2872
[I 2024-10-21 23:01:35,805] Trial 4 finished with value: 0.9733333333333333 and parameters: {'n_estimators': 7, 'max_depth': 3.57395
[I 2024-10-21 23:01:36,528] Trial 5 finished with value: 0.9466666666666667 and parameters: {'n_estimators': 9, 'max_depth': 5.23798
[I 2024-10-21 23:01:36,568] Trial 6 finished with value: 0.9533333333333333 and parameters: {'n_estimators': 16, 'max_depth': 7.9624
```

Optuna package - example



Advanced modelling topics

Interaction constraints

Two features interact within a model, if the second-order partial derivative is non-zero:

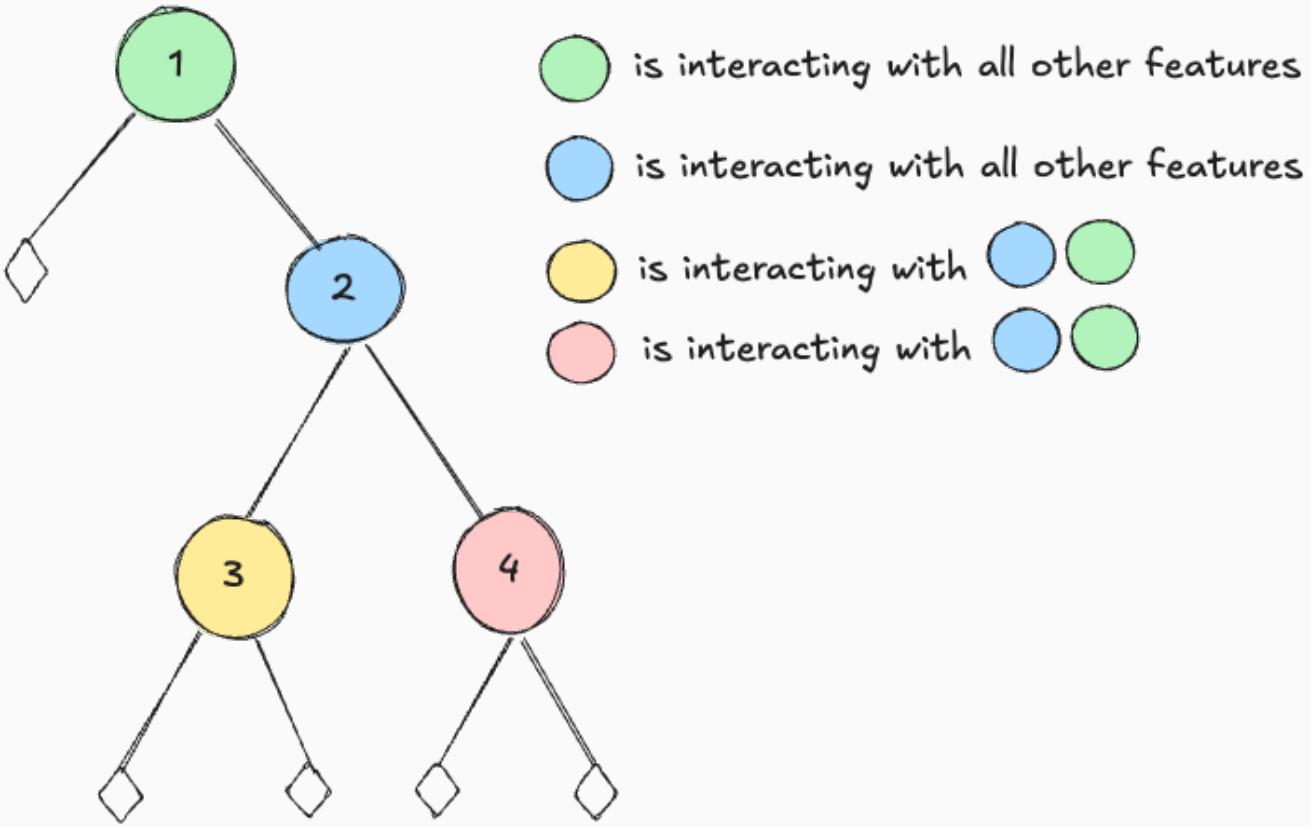
$$\frac{\delta^2 f}{\delta x_1 \delta x_2} \neq 0.$$

Sometimes we want to disallow specific interactions as we want the final model to be linear w.r.t. the marginal effect of some feature.

Parametric models: With parametric models any interaction constraint is implicit, as we simply don't include the interaction in the first place.

Tree based models: Trees are very eager in interacting features, hence we have to explicitly exclude interactions if we want to constrain them. Within a tree, features are only interacting when they appear in the same branch.

Interaction constraints



Monotonicity constraints

We sometimes want to constrain our model to follow a specific monotonicity with respect to specific numeric or ordinal categoricals. In essence, this is also a way of regularizing our model using domain knowledge.

Example

Take again the mileage example: For the same risk factors, we expect the risk of a car accident to increase the more miles/km a person drives. Yet, with too little data in some segments, we might empirically observe a break of this monotonicity. Hence, we use a monotonicity constraint to tell the model that the marginal effect should be monotonically increasing.

Monotonicity constraints

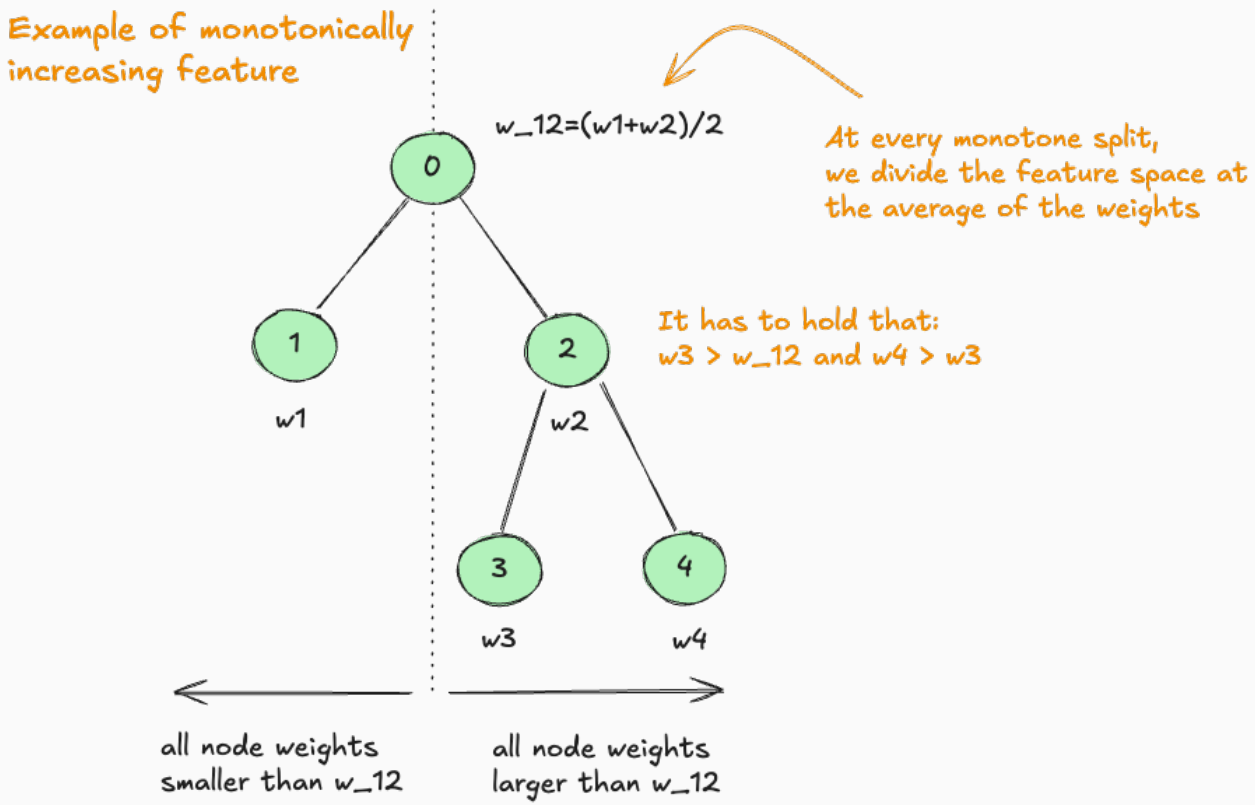
How are monotonicity constraints implemented?

Parametric models: In parametric models, we would have to make sure the coefficients represent the monotonicity, either in their order, if the feature is dummy encoded, or in the signs, e.g. when the feature is modelled as a spline or polynomial.

Tree based models: Tree based models offer a quite native structure to implement monotonicity constraints. We would only split, if the split obeys the monotonicity. Thereby we do not even form any interactions of splits which would undermine the monotonicity. Given the additivity of trees when boosting or in a random forest, the monotonicity also holds globally.

Monotonicity constraints

While this simple approach of constraining every further split by the mean of the splits before, seems overly restrictive, it works well in practice. There are also less restrictive implementations available.



How to introduce monotonicity constraints in LGBM

Using the learning parameter `monotone_constraint`, we can provide a list of feature specific constraints.

Without any convenience wrapper, we have to specify all features in their order: `[-1, 0, 1]`, where -1 means decreasing in the 1st feature, 0 no monotonicity constraint in the 2nd feature and 1 increasing in the 3rd feature.

For details, see: LGBM parameter documentation

Custom loss function in LGBM

In real-world industry setting we can face very specific loss functions far beyond the typical textbook loss functions for specific distributions.

Example

For instance, we might face a client-specific profit function for which we are asked to find an optimal price function. This can either be achieved by setting up and optimizing the problem parametrically, or we could specify the custom loss function and its derivatives and boost based on some feature space.

For examples for conditional distribution predictions see: LightGBMLSS

Custom loss function in LGBM - MSE example

```python
import numpy as np

# Define the custom loss function
def custom_mse_loss(preds, data):
    gradient = (preds - data.get_label())
    hessian = np.ones_like(preds)
    return gradient, hessian

# Wrapper function for LightGBM
def custom_mse_eval(preds, data):
    loss = (preds - data.get_label())**2
    return "mse", loss.mean(), False

# Generate some sample data
X_train = np.array(np.random.uniform(size=1000)).reshape(-1, 1)
y_train = 2*X_train

import lightgbm as lgb

# Train LightGBM models using default and custom loss functions
params_default = {'objective': 'regression', 'metric': 'mse'}
params_custom = {'objective': custom_mse_objective, 'metric': 'mse'}

lgb_train = lgb.Dataset(X_train, y_train)

model_default = lgb.train(params_default, lgb_train, num_boost_round=100)
model_custom = lgb.train(params_custom, lgb_train, num_boost_round=100)

# Make predictions
y_pred_default = model_default.predict(X_train)
y_pred_custom = model_custom.predict(X_train)

# Assert predictions are the same
assert all(np.isclose(y_pred_default, y_pred_custom, atol=0.0001))
```

✓ 2.4s

We have to define the gradient and hessian for the booster to know the direction to improve in. Note that this is invariant to any factor as taking logs (log-loss) still provides the same optimization trajectory.

We also define an evaluation function for the custom loss and define whether higher is better, here False

now we can provide our custom loss function to the 'objective' argument

we check that the custom objective gives us the same results as the default one by some small tolerance

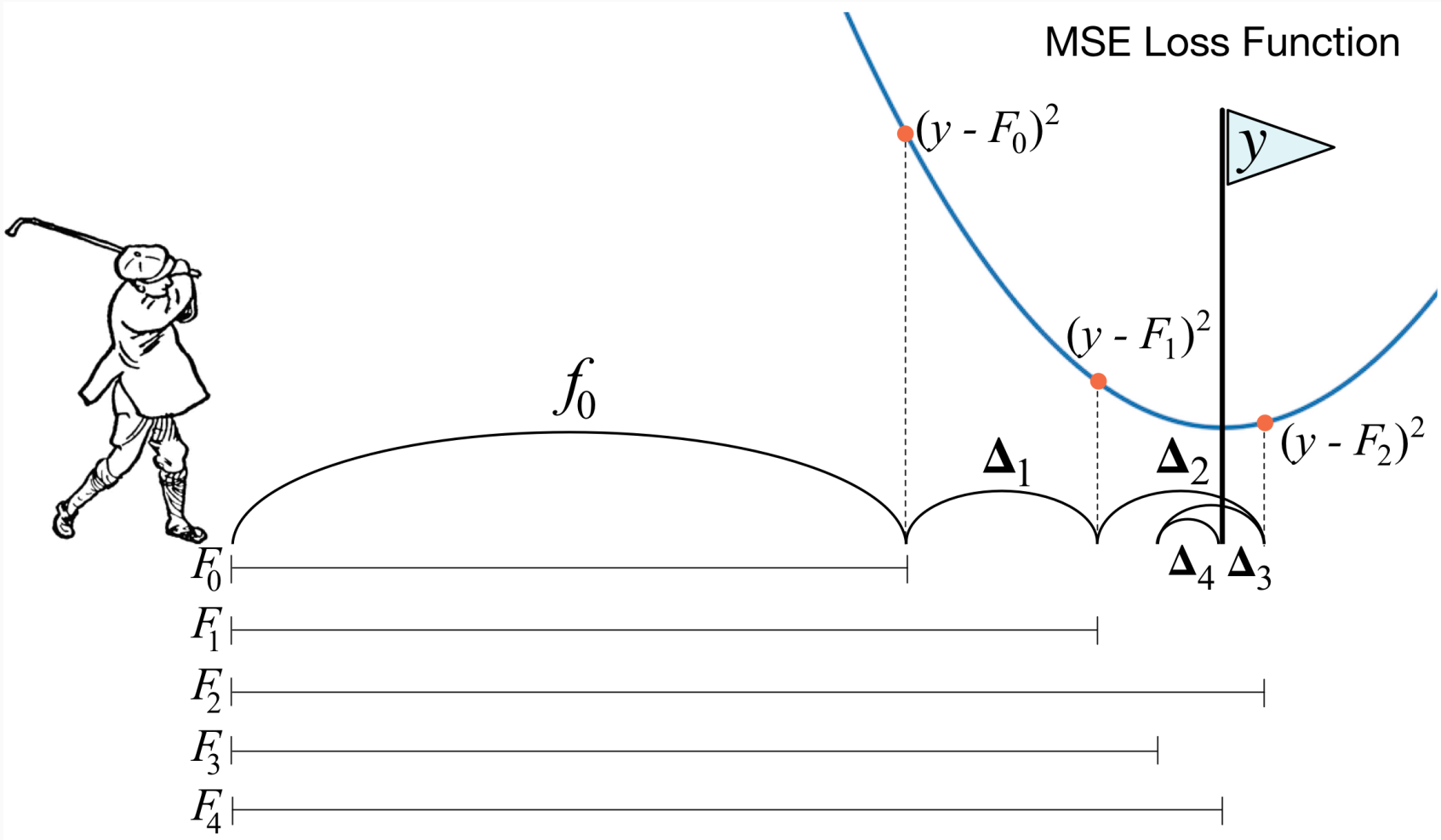
Custom loss function in LGBM - Intuition

Every convex, 2-times differentiable loss function can be minimized using GBMs.

Note

Note how for MSE, the gradient coincides with the residuals and the hessian is simply a vector of ones. In most textbooks, GBM are explained by simply updating the residuals of the tree iterations before, yet the "Gradient" in "Gradient Boosting" is coming from the fact, that we are moving along the gradient of the loss function and the hessian is informing us by how much we should optimally move. If the hessian is large, i.e. the gradient is changing strongly, we move slower than if the hessian is small and we expect a similar gradient around, which corresponds to Newton's Method (also Newton-Raphson) for root-finding.

Custom loss function in LGBM - Intuition



Source: explained.ai blog on GBMs

Bibliography

An Introduction to Statistical Learning, with Applications in Python

Gini score in ML

LGBM docs on parameter tuning

LGBM Tuner in optuna

Custom Loss LGBM example

Notes on GLMs

Glum library with various tutorials